Lecture Summary- PBQ-01 and PBQ-02

Python Basics

Important Facts about Python:

Python is dynamically data typed. It identifies the datatype of the variable as per the data contained in it.
Python does not need variable declaration.
Python is case sensitive.
Python doesn't require semi colon to conclude the statements.
Python uses indentation to highlight the line/blocks of code.

Variable Declaration, Data Type, Conversions and Operations

Data Type category	Data types	Defined as (examples)	Type conversion	Operations

Numeric	Integer	`x=9`	`int(6.8) --> 6`	`x=6.8,y=5`
				`z=x+y`
				`z=x/y,z=x-y,z=x*y`
				`type(x)`

	Float	`y=8.56`	`float(8)-->8.0`
			`float("-31.54e8") → -3154000000.0`
			`float(-31.54e8) → -3154000000.0`
				`type(y)`

Sequence	List	`lst1=[1,'string',9.67]`	`lst2=list((1,'string',9.67))`	`lst1.append(lst_item)`
				`lst1.remove(lst_item)`
				`lst1.insert(index,lst_item)`
				`lst1.sort(lst_item) # to be used for int and float`
				`lst1==lst2`
				`type(lst1)`

	Tuple	`tup = ("1", "banana", True, 3-5j)`	`tup = ("1", "banana", True, 3-5j)`	`type(tup)` `No operations since a tuple cannpt be modified (immutable).`

	Dictionary	`dct = {`
		`"Open": 100,`
		`"High": 108,`
		`"Low": 98,`
		`"Close": 102`
		`}`
			`dct([(2,"two"),(4,"four")]) → {2:'two',4:'four'}`	`type(dct)`

String/text	String	`txt='let us learn Python'`	`txt=str('let us learn Python')`
			`num_to_str=str(2)`
				`verify=('me' in txt)`
				`verify = 'me' not in txt`
				`txt.upper(),`
				`txt.lower()`
				`txt.strip(),`
				`txt[1:5]`
				`txt.find()`
				`txt.split()`
				`txt.startswith()`
				`txt.endswith()`
				`type(txt)`

Boolean	Boolean	`x = 0 is equivalent to x=False`
		`y = 1 is equivalent to y=True`
		`a = False`
		`b = True`
				`z = x & y`
				`z = x | y`
				`c = a & b`
				`c = a | b`

Conditional Expressions and Statements

Conditional Exp/Stmt	Syntax and return value	Examples
Conditional Expression	returns boolean value True/False	`2>3 returns False`

Conditional Statement	`if condition_expression_1:`
	`  statements_block_1`
	`elif condition_expression_2:`
	`  statements_block_2`
	`else condition_expression_2:`
	`  statements_block_2`
		`people = 10`
		`sandwiches = 12`
		`coffees = 8`
		
		`if coffees > people:`
		`  print("That's too much coffee")`
		`elif coffees < people:`
		`  print("Too many people!")`
		`else:`
		`  print("Just enough for everyone.")`

Loop Statements

Loop Exp/Stmt	Syntax and return value
for	*`for iter_var in the range:`* *`  statements_block_1`* *`  if condition_expression_1:`* *`  break(exit loop)`*
while	*`while condition_expression_1:`* *`  statements_block_1`* *`  if condition_expression_1:`* *`  break(exit loop)`*

Functions

File Operations

Function type	Declaration	Call Function	Operations	Syntax
Function not Returning any Value	def func_quant(a, b, c=10): statements Note: input var c has has default value 10	*func_quant(a=5, b=2, c=12)* Note: The value of c can be overwritten	Open/Close	Open: *1. fname = open("filename.txt","w")* *2. with open("filename.txt","w") as fname* *file_operations* Close: *fname.close()*

Function Returning a Value	def func_quant(a, b):
	  statements
	  return z
		z=func_quant(a=5, b=2, c=12)	Read/write	fname.write("Quantitative_analysis")
				fname.writelines(list of lines)
				
				fname.read(n)
				fname.readlines(n)

Pandas DataFrame

Creating and Importing Data in Pandas Dataframe

Source Category	Source	Example

In-built Objects	List	import pandas as pd
		
		# List of dictionary
		list_dict_data = [{'x': 3, 'y': 1},
		{'x': 2, 'y': 5, 'z': 10}]
		
		# Dictionary keys become columns
		df_list_dict = pd.DataFrame(list_dict_data)

	Dictionary	import pandas as pd
		
		dict_list_data = {'id':['a', 'b', 'c', 'd'],
		'score':[99, 98, 95, 90],
		'age':[23,24,26,21]}
		
		# Dictionary keys become columns
		df_dict_list = pd.DataFrame(dict_list_data)

External Source	CSV	df=pd.read_csv("file.csv")
	Excel	df=pd.read_xlsx("file.xlsx")
	HTML	df=pd.read_html("file.html") df=pd.read_html("URL")
	Clipboard	df=pd.read_clipboard(sep)
	JSON	df=pd.read_json("file.json")

Assigning and Accessing Pandas Dataframe Data

Operation	Example
Accessing the column of a dataframe	df_list_dict['x']
Accessing multiple columns of a dataframe	df_dict_list[['id','score']]
Assigning value of a series to a column	df['sum_AB']=df['A']+df['B']
Accessing value of a row and column, with row id	s=df['A'].iloc[5]
Accessing value of a row and column, with row label	s=df['A'].loc['2021-04-10']
Assigning value of a row and column, with row id	df['A'].iat[5]=s
Assigning value of a row and column, with row lable	df['A'].at['2021-04-10']=s

Numpy Arrays

Creating Numpy Array and Importing Data in it

Source Category	Source	Example

Inbuilt objects	List	import numpy as np
		
		#conversion from list to numpy array
		a = np.asarray([1,2,3])
		
		'''Output : array([1, 2, 3])'''

Mathematical, Comparison and Conditional Operations

Operation	Example
Mathematical operations	a=np.asarray([1,2,3,4,5]) b=np.asarray([2,3,4,5,6]) c_add=b+a '''Output : array([3, 5, 7, 9, 11]) ''' c_subt=b-a '''Output : array([1, 1, 1, 1, 1]) ''' c_mult=b*a '''Output : array([2, 6, 12, 20, 30]) ''' c_div=b/a '''Output : array([2. , 1.5 , 1.33333333, 1.25 , 1.2]) ''' c_exp=np.exp(a) '''Output : array([2.71828183,7.3890561 , 20.08553692,54.59815003,148.4131591]) ''' c_sqrt=np.sqrt(a) '''Output : array([1., 1.41421356, 1.73205081, 2., 2.23606798])''' c_log=np.log(a) '''Output : array([0., 0.69314718, 1.09861229, 1.38629436, 1.60943791])'''

	Tuple	import numpy as np
		
		#conversion from tuple to numpy array
		a = np.asarray([(1,3,5,7), (2,4,6,8)], dtype = float)
		
		'''Output : array([[1., 3., 5., 7.],
		[2., 4., 6., 8.]])'''
		
		a=np.asarray((1,2,3,4,5),dtype=float)
		
		'''Output : array([1., 2., 3., 4., 5.])'''
			Comparison and Conditional Operations	c_is_equal=(a == b)
				'''Output : array([False, False, False, False, False]) '''
				c_is_greater=(a > b)
				'''Output : array([False, False, False, False, False]) '''
				c_is_lesser=(a < b)
				'''Output : array([True, True, True, True, True]) '''
				
				#c_conditional=np.where(condition,value_if_condition_is_True,
				value_if_condition_is_False)
				
				c_conditional_1=np.where(((b-a)/a)>=1,b,a)
				'''Output : array([2, 2, 3, 4, 5]) '''
				c_conditional_2=np.where(((b-a)/a)<=1,a,b)
				'''Output : array([1, 2, 3, 4, 5]) '''